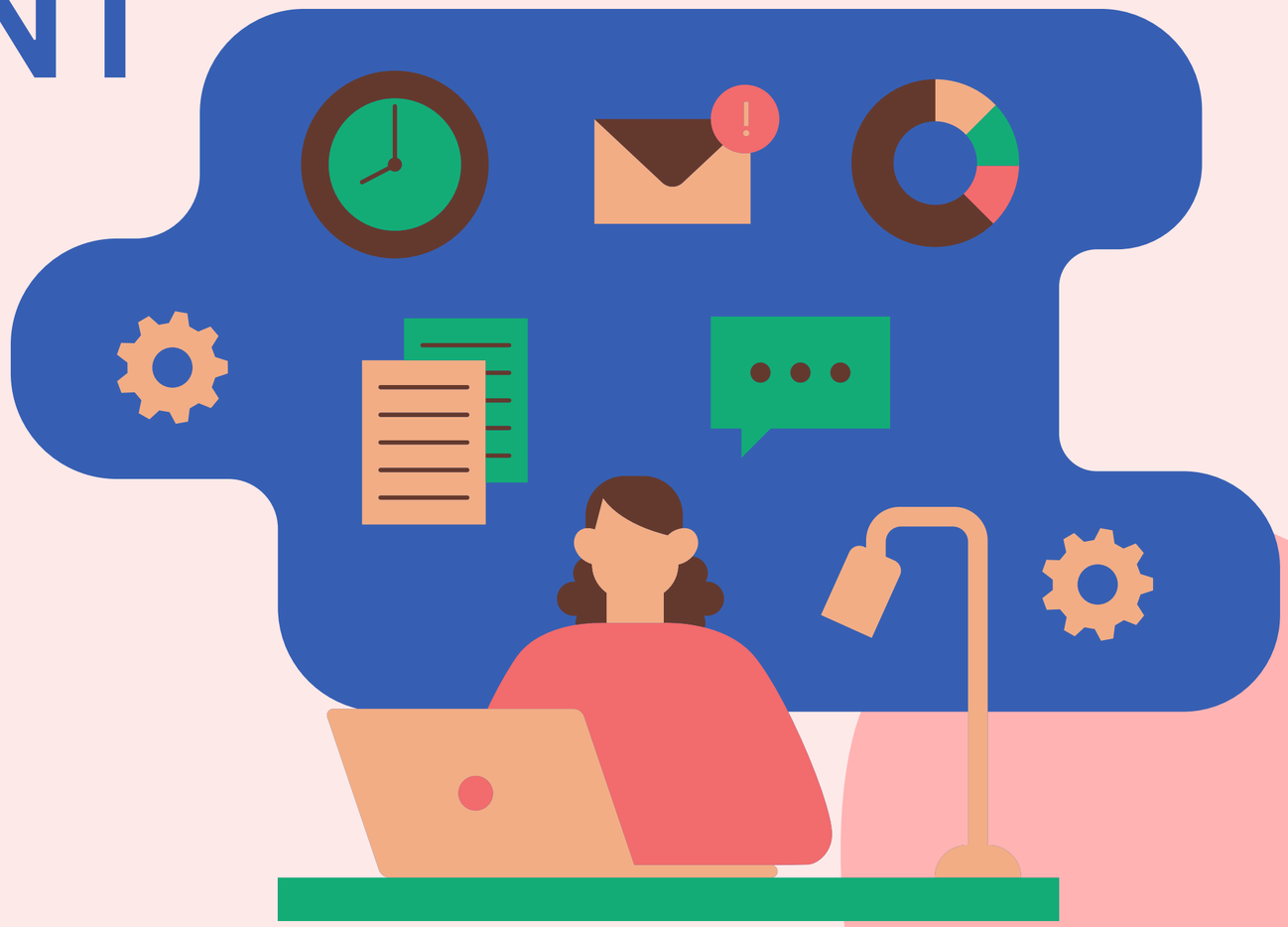
STUDENT PROFILE MANAGEMENT SYSTEM



Presentation by:
Adarsha Karki
Bishesh Thapa
Jonisha Ghimire

Presentation overview

1. Project introduction and objectives
2. System architecture and OOP design
3. Key features — Admin and Student roles
4. Data storage — 4 CSV files
5. Analytics dashboard (Task 2 bonus)
6. Flowcharts and algorithm walkthrough
7. Testing and results
8. Challenges and what we learned

INTRODUCTION

This is an terminal based application

Consists of two roles:

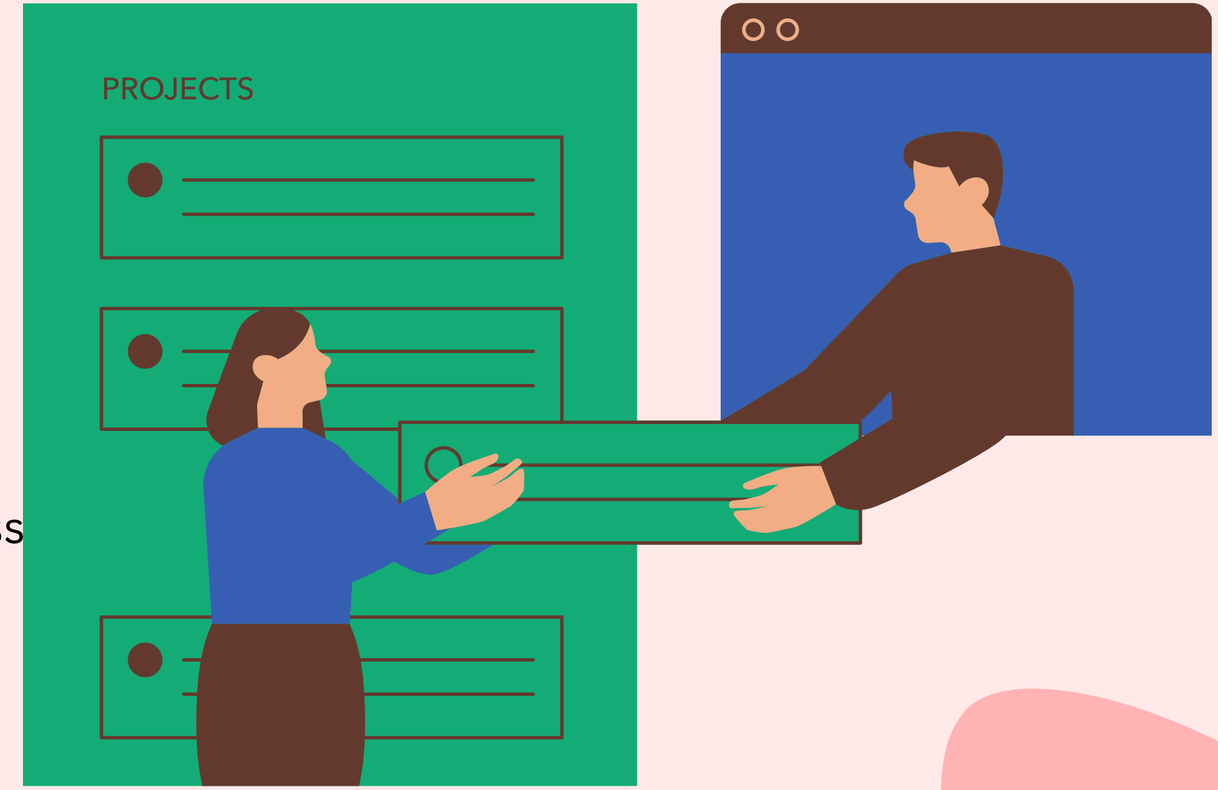
- ADMIN (full control)
- STUDENT (view and update own data)

Data stored in 4 CSV files (passwords.csv, users.csv, grades.csv, eca,csv)

An analytics dashboard shows performance trends using pandas, numpy, and matplotlib.

SYSTEM DESIGN

Core System Modules



OOP Concepts:

- **Inheritance:** Student & Admin extend User
- **Encapsulation:** FileHandler manages data access
- **Abstraction:** Panels hide system complexity

Design Principle:

- **Modularity:** Each class has single responsibility
- **Exception Handling:** Used for error handling and system stability (try/except)

FEATURES

ADMIN PANEL

- Manage students
  (Add, View, Update, Delete)
- Manage grades & ECA
- View analytics & performance alerts

STUDENT PANEL

- View profile, grades and ECA
- Update personal info (email, phone)
- View grade charts (text / matplotlib)

DATA STORAGE & ANALYTICS

DATA STORAGE

- **users.csv:** user information
- **passwords.csv:** login credentials
- **grades.csv:** academic records
- **eca.csv:** extracurricular activities

Analytics Dashboard:

- **Grade trends visualization**
- **ECA performance analysis**
- **Performance alerts for low scores**

FOLDER STRUCTURE

```
student_system/
│
├── main.py
├── auth.py
├── models.py
├── admin.py
├── student.py
├── analytics.py
├── file_handler.py
├── utils.py
│
├── data/
│   ├── users.csv
│   ├── passwords.csv
│   ├── grades.csv
│   └── eca.csv
│
└── requirements.txt
```

CHALLENGES WE FACED

- Keeping multiple CSV files in sync during add/delete operations
- Understanding pandas DataFrame vs Python dictionaries
- Correct role-based routing (Admin / Student menus)
- Generating meaningful analytics with small sample data
- Using NumPy functions correctly (need enough data points)

WHAT WE LEARNED

- OOP concepts using inheritance
- Data handling using pandas
- Numerical operations using NumPy
- Data visualization using matplotlib
- Error handling using exception handling
- Basic role-based system design

Conclusion

- Built a working Student Management System
- Used OOP, file handling, pandas, and NumPy
- Implemented role-based access and analytics
- Improved Python programming and system design skills